

AI ASSISTED CODING ASSIGNMENT – 9.5

CH. NAKSHATRA (2303A51094)

BATCH-02

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):  
    return text[::-1]
```

Task:

1. Write documentation in:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

```
Programs > Python > AIAC > demo.py > ...  
1  C:\Users\DELL\OneDrive\Desktop\Programs  
2  """  
3      Docstring for the reverse_string function.  
4      :param text: The string to be reversed.  
5      :return: The reversed string.  
6      """  
7      return text[::-1]  
8  if __name__ == "__main__":  
9      input_string = "Hello, World!"  
10     reversed_string = reverse_string(input_string)  
11     print(f"Original string: {input_string}")  
12     print(f"Reversed string: {reversed_string}")  
13
```

```
def reverse_string(text):  
    return text[::-1] # This line reverses the input string by slicing it with a step of -1.  
if __name__ == "__main__":  
    input_string = "Hello, World!" # This is the string that we want to reverse.  
    reversed_string = reverse_string(input_string) # We call the reverse_string function and store the  
    # result in the variable reversed_string.  
    print(f"Original string: {input_string}") # This line prints the original string.  
    print(f"Reversed string: {reversed_string}") # This line prints the reversed string.
```

```
def reverse_string(text: str) → str:
    """
    Docstring for the reverse_string function.

    :param text: The string to be reversed.
    :type text: str
    :return: The reversed string.
    :rtype: str
    :exceptions: None
    :error handling: None
    :side effects: None
    :raises: None
    """
    return text[::-1]
if __name__ == "__main__":
    input_string = "Hello, World!"
    reversed_string = reverse_string(input_string)
    print(f"Original string: {input_string}")
    print(f"Reversed string: {reversed_string}")
```

Output:

```
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\
Original string: Hello, World!
Reversed string: !dlroW ,olleH
Original string: Hello, World!
Reversed string: !dlroW ,olleH
PS C:\Users\DELL\OneDrive\Desktop\Programs> █
```

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

Output:

Problem 3: Math Utilities Module

Instructions:

1. Create a module `math_utils.py` with functions:
`square(n)`
`cube(n)`
`factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

Output:

Problem 4: Attendance Management Module

Task:

1. Create a module `attendance.py` with functions:
`mark_present(student)`
`mark_absent(student)`
`get_attendance(student)`
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

Output:

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.